

Titanic Survival Prediction using Logistic Regression

Introduction

This project uses Logistic Regression to predict whether a passenger survived the Titanic disaster.

The project includes data loading, data exploration, preprocessing, feature selection, train-test splitting, model training, prediction, probability prediction, and model evaluation.

```
In [1]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
In [5]: df = pd.read_csv("train.csv")

df.head()
```

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500

In [6]:

```
print("Shape of dataset:", df.shape)
```

Shape of dataset: (891, 12)

In [7]:

```
df.info()
df.describe()
df.isnull().sum()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age         714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    str
9   Fare         891 non-null    float64
10  Cabin        204 non-null    str
11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

```
Out[7]: PassengerId      0
        Survived        0
        Pclass          0
        Name            0
        Sex             0
        Age            177
        SibSp           0
        Parch           0
        Ticket          0
        Fare            0
        Cabin          687
        Embarked        2
        dtype: int64
```

```
In [8]: df["Survived"].value_counts()
```

```
Out[8]: Survived
0      549
1      342
Name: count, dtype: int64
```

```
In [9]: df["Survived"].value_counts(normalize=True)
```

```
Out[9]: Survived
0      0.616162
1      0.383838
Name: proportion, dtype: float64
```

```
In [10]: features = [
        "Pclass",
        "Sex",
        "Age",
        "SibSp",
        "Parch",
        "Fare",
        "Embarked"
    ]

X = df[features].copy()
y = df["Survived"]
```

```
In [11]: X["Age"] = X["Age"].fillna(X["Age"].median())
X["Fare"] = X["Fare"].fillna(X["Fare"].median())
X["Embarked"] = X["Embarked"].fillna(X["Embarked"].mode()[0])
```

```
In [12]: X.isnull().sum()
```

```
Out[12]: Pclass      0
        Sex         0
        Age         0
        SibSp       0
        Parch       0
        Fare        0
        Embarked    0
        dtype: int64
```

```
In [13]: X = pd.get_dummies(
        X,
        columns=["Sex", "Embarked"],
        drop_first=True
    )

X.head()
```

```
Out[13]:
```

	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	True	False	True
1	1	38.0	1	0	71.2833	False	False	False
2	3	26.0	0	0	7.9250	False	False	True
3	1	35.0	1	0	53.1000	False	False	True
4	3	35.0	0	0	8.0500	True	False	True

```
In [14]: X.dtypes
```

```
Out[14]: Pclass      int64
Age          float64
SibSp        int64
Parch        int64
Fare         float64
Sex_male      bool
Embarked_Q    bool
Embarked_S    bool
dtype: object
```

```
In [15]: X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )

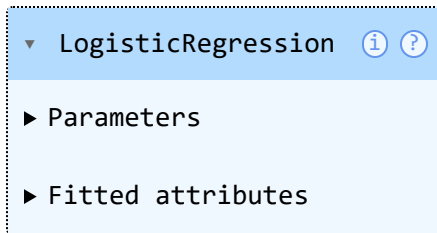
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (712, 8)
Testing data: (179, 8)
```

```
In [17]: model = LogisticRegression(max_iter=1000)
```

```
In [19]: model.fit(X_train, y_train)
```

Out[19]:

In [20]: `y_pred = model.predict(X_test)`

```
print("Predictions:")
print(y_pred)
```

Predictions:

```
[0 0 0 1 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 1 0 0 0 0 0 1 0 1 1 0 0 0
 1 1 0 0 0 0 0 1 0 0 0 0 0 1 1 0 1 0 1 0 1 1 1 0 1 1 0 0 1 0 0 0 1 1 1 1
 0 0 1 1 1 0 0 1 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1
 0 1 0 1 0 0 0 1 0 0 1 1 0 0 1 1 1 1 0 1 0 0 1 0 1 1 0 0 1 0 1 0 0 0 1 0 0
 1 0 0 0 0 1 0 0 0 1 1 1 0 0 0 1 0 0 0 1 0 0 1 1 0 1 0 0 0 1 1]
```

In [21]: `accuracy = accuracy_score(y_test, y_pred)`

```
print("Model Accuracy:", accuracy)
print("Model Accuracy (%)ate:", accuracy * 100)
```

Model Accuracy: 0.8100558659217877

Model Accuracy (%): 81.00558659217877

In [22]: `y_probability = model.predict_proba(X_test)`

```
print("Probability Predictions:")
print(y_probability[:10])
```

Probability Predictions:

```
[[0.89000888 0.10999112]
 [0.77786006 0.22213994]
 [0.86543114 0.13456886]
 [0.11373855 0.88626145]
 [0.26359019 0.73640981]
 [0.06875179 0.93124821]
 [0.31600646 0.68399354]
 [0.90921227 0.09078773]
 [0.24249543 0.75750457]
 [0.07816499 0.92183501]]
```

```
In [23]: results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred,
    "Survival Probability": y_probability[:, 1]
})

results.head(10)
```

Out[23]:

	Actual	Predicted	Survival Probability
0	1	0	0.109991
1	0	0	0.222140
2	0	0	0.134569
3	1	1	0.886261
4	1	1	0.736410
5	1	1	0.931248
6	1	1	0.683994
7	0	0	0.090788
8	1	1	0.757505
9	1	1	0.921835

```
In [24]: from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)

print("===== MODEL EVALUATION =====")
print("Accuracy:", accuracy)
print("Accuracy Percentage:", accuracy * 100, "%")

===== MODEL EVALUATION =====
Accuracy: 0.8100558659217877
Accuracy Percentage: 81.00558659217877 %
```

```
In [25]: print("===== ACTUAL VS PREDICTED =====")

results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print(results)
```

===== ACTUAL VS PREDICTED =====

	Actual	Predicted
0	1	0
1	0	0
2	0	0
3	1	1
4	1	1
..	...	...
174	0	0
175	0	0
176	1	0
177	1	1
178	1	1

[179 rows x 2 columns]

```
In [26]: y_probability = model.predict_proba(X_test)

print("===== PREDICTION PROBABILITIES =====")
print(y_probability)
```

===== PREDICTION PROBABILITIES =====

```
[[0.89000888 0.10999112]
 [0.77786006 0.22213994]
 [0.86543114 0.13456886]
 [0.11373855 0.88626145]
 [0.26359019 0.73640981]
 [0.06875179 0.93124821]
 [0.31600646 0.68399354]
 [0.90921227 0.09078773]
 [0.24249543 0.75750457]
 [0.07816499 0.92183501]
 [0.66477697 0.33522303]
 [0.93051004 0.06948996]
 [0.61380089 0.38619911]
 [0.85430395 0.14569605]
 [0.80212641 0.19787359]
 [0.06288622 0.93711378]
 [0.69823867 0.30176133]
 [0.31596223 0.68403777]
 [0.75482102 0.24517898]
 [0.66765626 0.33234374]
 [0.87905528 0.12094472]
 [0.59047597 0.40952403]
 [0.38559522 0.61440478]
 [0.86503945 0.13496055]
 [0.88980355 0.11019645]
 [0.93133016 0.06866984]
 [0.53654099 0.46345901]
 [0.77674182 0.22325818]
 [0.91862974 0.08137026]
 [0.40260646 0.59739354]
 [0.86183979 0.13816021]
 [0.33861039 0.66138961]
 [0.44321524 0.55678476]
 [0.38094345 0.61905655]
 [0.85811062 0.14188938]
 [0.87588006 0.12411994]
 [0.53975995 0.46024005]
 [0.31600646 0.68399354]
 [0.05952402 0.94047598]
 [0.89146574 0.10853426]
 [0.79834044 0.20165956]
 [0.91939691 0.08060309]
 [0.89142718 0.10857282]
 [0.86057268 0.13942732]
 [0.43161742 0.56838258]
 [0.90813763 0.09186237]
 [0.86175901 0.13824099]
 [0.87539296 0.12460704]
 [0.87928744 0.12071256]
 [0.66647737 0.33352263]
 [0.28509745 0.71490255]
 [0.19076041 0.80923959]
 [0.95890408 0.04109592]
 [0.48339898 0.51660102]
 [0.9503206 0.0496794 ]]
```

[0.06685624 0.93314376]
[0.78309777 0.21690223]
[0.06915566 0.93084434]
[0.22376722 0.77623278]
[0.25967368 0.74032632]
[0.86899605 0.13100395]
[0.1413377 0.8586623]
[0.23592418 0.76407582]
[0.54589989 0.45410011]
[0.86057268 0.13942732]
[0.36722596 0.63277404]
[0.68669884 0.31330116]
[0.89439357 0.10560643]
[0.85511026 0.14488974]
[0.12853746 0.87146254]
[0.24453495 0.75546505]
[0.05132624 0.94867376]
[0.42862367 0.57137633]
[0.09834944 0.90165056]
[0.87577882 0.12422118]
[0.93252938 0.06747062]
[0.33275375 0.66724625]
[0.05542344 0.94457656]
[0.24389194 0.75610806]
[0.52393589 0.47606411]
[0.95014815 0.04985185]
[0.19779314 0.80220686]
[0.08965992 0.91034008]
[0.86057655 0.13942345]
[0.65200986 0.34799014]
[0.73843709 0.26156291]
[0.06621002 0.93378998]
[0.05588211 0.94411789]
[0.63571759 0.36428241]
[0.89162709 0.10837291]
[0.84670056 0.15329944]
[0.56743224 0.43256776]
[0.76431048 0.23568952]
[0.86057912 0.13942088]
[0.89146574 0.10853426]
[0.89147614 0.10852386]
[0.73570962 0.26429038]
[0.93246101 0.06753899]
[0.2327938 0.7672062]
[0.88847219 0.11152781]
[0.78321595 0.21678405]
[0.91066608 0.08933392]
[0.04945712 0.95054288]
[0.91786636 0.08213364]
[0.89860612 0.10139388]
[0.90970265 0.09029735]
[0.32441341 0.67558659]
[0.67535088 0.32464912]
[0.89685879 0.10314121]
[0.53443332 0.46556668]
[0.10157114 0.89842886]

[0.86740215 0.13259785]
[0.06442388 0.93557612]
[0.60533749 0.39466251]
[0.4201968 0.5798032]
[0.86485206 0.13514794]
[0.74578049 0.25421951]
[0.73106789 0.26893211]
[0.22131593 0.77868407]
[0.66310448 0.33689552]
[0.84626851 0.15373149]
[0.05647123 0.94352877]
[0.2356005 0.7643995]
[0.62239719 0.37760281]
[0.88244132 0.11755868]
[0.49914791 0.50085209]
[0.14602467 0.85397533]
[0.3832576 0.6167424]
[0.35258439 0.64741561]
[0.86891513 0.13108487]
[0.31593431 0.68406569]
[0.89000888 0.10999112]
[0.76650055 0.23349945]
[0.23122238 0.76877762]
[0.65513954 0.34486046]
[0.26793586 0.73206414]
[0.04619593 0.95380407]
[0.89441797 0.10558203]
[0.92653883 0.07346117]
[0.3723526 0.6276474]
[0.86880228 0.13119772]
[0.33492154 0.66507846]
[0.76043465 0.23956535]
[0.84451713 0.15548287]
[0.51262096 0.48737904]
[0.28900997 0.71099003]
[0.77139543 0.22860457]
[0.91078438 0.08921562]
[0.05603876 0.94396124]
[0.92850703 0.07149297]
[0.84602743 0.15397257]
[0.87916512 0.12083488]
[0.85819058 0.14180942]
[0.37309745 0.62690255]
[0.89142718 0.10857282]
[0.89167702 0.10832298]
[0.87870458 0.12129542]
[0.31598552 0.68401448]
[0.28280652 0.71719348]
[0.43389223 0.56610777]
[0.82778893 0.17221107]
[0.54549951 0.45450049]
[0.81628127 0.18371873]
[0.06551219 0.93448781]
[0.86035293 0.13964707]
[0.71718082 0.28281918]
[0.73275884 0.26724116]

```
[0.10160356 0.89839644]
[0.87240256 0.12759744]
[0.91068185 0.08931815]
[0.41167231 0.58832769]
[0.17854385 0.82145615]
[0.60527576 0.39472424]
[0.25307637 0.74692363]
[0.85464457 0.14535543]
[0.84652746 0.15347254]
[0.64469746 0.35530254]
[0.1459151  0.8540849 ]
[0.30175081 0.69824919]]
```

```
In [27]: probability_results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred,
    "Probability_Not_Survived": y_probability[:, 0],
    "Probability_Survived": y_probability[:, 1]
})

print("=====PREDICTION RESULTS=====")
print(probability_results)
```

```
=====PREDICTION RESULTS=====
```

	Actual	Predicted	Probability_Not_Survived	Probability_Survived
0	1	0	0.890009	0.109991
1	0	0	0.777860	0.222140
2	0	0	0.865431	0.134569
3	1	1	0.113739	0.886261
4	1	1	0.263590	0.736410
..	...	...	...	...
174	0	0	0.854645	0.145355
175	0	0	0.846527	0.153473
176	1	0	0.644697	0.355303
177	1	1	0.145915	0.854085
178	1	1	0.301751	0.698249

```
[179 rows x 4 columns]
```

```
In [28]: print("Features used by the model:")
print(X.columns.tolist())
```

Features used by the model:

```
['Pclass', 'Age', 'SibSp', 'Parch', 'Fare', 'Sex_male', 'Embarked_Q', 'Embarked_S']
```

```
In [29]: # New passenger data

new_passenger = pd.DataFrame({
    "Pclass": [3],
    "Age": [22],
    "SibSp": [0],
    "Parch": [0],
    "Fare": [7.25],
    "Sex_male": [1],
    "Embarked_Q": [0],
    "Embarked_S": [1]
})
```

```

# Make prediction
new_prediction = model.predict(new_passenger)

# Get prediction probability
new_probability = model.predict_proba(new_passenger)

print("===== NEW PASSENGER PREDICTION =====")

if new_prediction[0] == 1:
    print("Prediction: Passenger is likely to SURVIVE")
else:
    print("Prediction: Passenger is likely to NOT SURVIVE")

print("Probability of NOT SURVIVING:", new_probability[0][0])
print("Probability of SURVIVING:", new_probability[0][1])

```

```

===== NEW PASSENGER PREDICTION =====
Prediction: Passenger is likely to NOT SURVIVE
Probability of NOT SURVIVING: 0.8725882052152418
Probability of SURVIVING: 0.12741179478475817

```

```

In [30]: print("===== PASSENGER DETAILS =====")
print("Passenger Class:", new_passenger["Pclass"].iloc[0])
print("Age:", new_passenger["Age"].iloc[0])
print("Sex:", "Male" if new_passenger["Sex_male"].iloc[0] == 1 else "Female")
print("Fare:", new_passenger["Fare"].iloc[0])

print("\n===== FINAL PREDICTION =====")

if new_prediction[0] == 1:
    print("The model predicts that the passenger would SURVIVE.")
else:
    print("The model predicts that the passenger would NOT SURVIVE.")

```

```

===== PASSENGER DETAILS =====
Passenger Class: 3
Age: 22
Sex: Male
Fare: 7.25

===== FINAL PREDICTION =====
The model predicts that the passenger would NOT SURVIVE.

```

```

In [31]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

print("===== CONFUSION MATRIX =====")
print(cm)

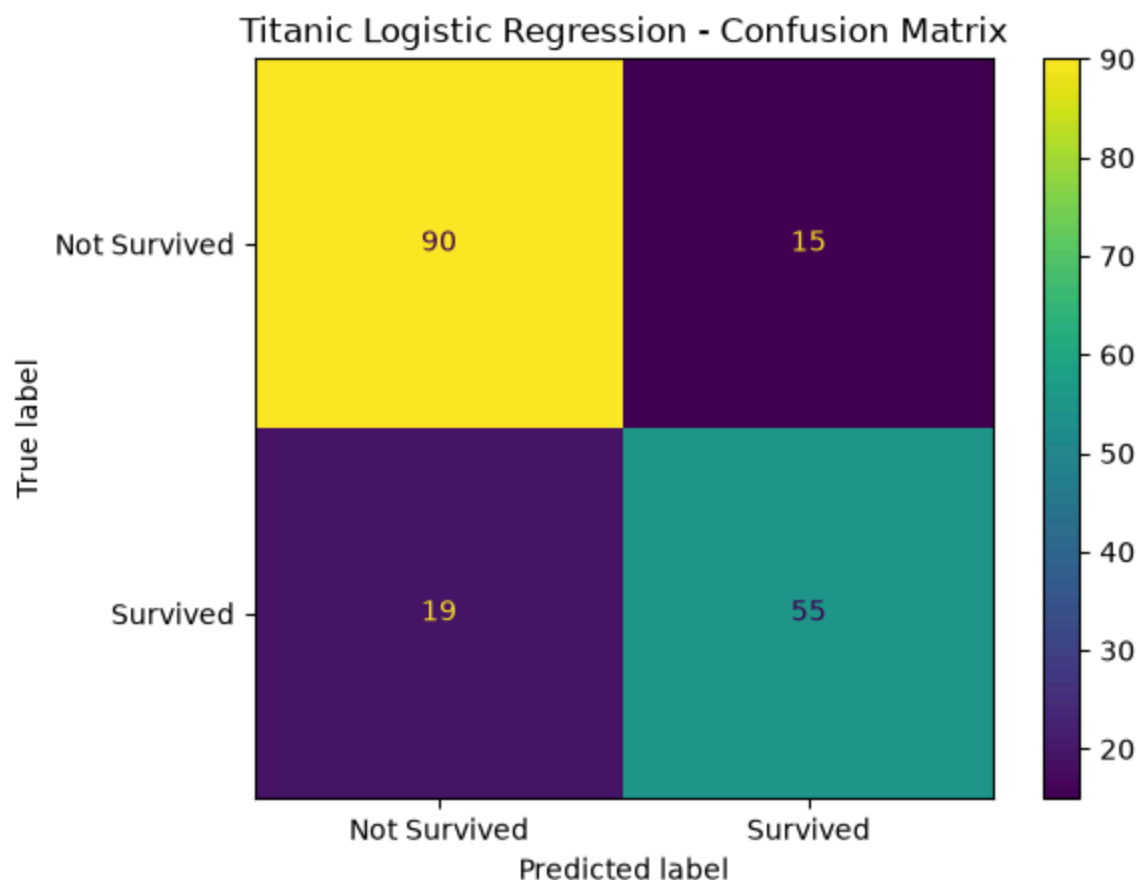
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not Survived", "Survived"]
)

```

```
disp.plot()
plt.title("Titanic Logistic Regression - Confusion Matrix")
plt.show()
```

===== CONFUSION MATRIX =====

```
[[90 15]
 [19 55]]
```



```
In [32]: from sklearn.metrics import classification_report

print("===== CLASSIFICATION REPORT =====")
print(classification_report(
    y_test,
    y_pred,
    target_names=["Not Survived", "Survived"]
))
```

===== CLASSIFICATION REPORT =====

	precision	recall	f1-score	support
Not Survived	0.83	0.86	0.84	105
Survived	0.79	0.74	0.76	74
accuracy			0.81	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

```
In [33]: print("=====")
print(" TITANIC LOGISTIC REGRESSION MODEL")
```

```
print("=====")

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
print("Accuracy:", round(accuracy * 100, 2), "%")

print("=====")
```

```
=====
      TITANIC LOGISTIC REGRESSION MODEL
=====
Training Samples: 712
Testing Samples: 179
Accuracy: 81.01 %
=====
```